Performance Analysis and Comparison of Classification Algorithms for Student Performance Analysis Prediction

Guide: Dr.Rambabu Pemula

Authors: Devara Sruthi(24911A6679)

Nandyala Snehitha (24911A66A5)

Peddabomma Srija (24911A66B3)

PROBLEM STATEMENT:

Student academic performance is influenced by several factors, including study time, previous grades, family background, attendance, failures, and other social and personal attributes. Identifying students who may achieve low academic performance at an early stage is important for educational institutions to provide appropriate guidance and support.

The main objective of this project is to develop a machine learning-based system that analyzes student-related data and predicts their academic performance. The project uses the Student Performance Dataset (student-mat.csv) obtained from Kaggle, which contains various academic, personal, and social attributes of students.

Different machine learning algorithms, including K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Decision Tree, and Logistic Regression, will be implemented and evaluated using the same dataset. Their performance will be compared using evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

Based on the evaluation results, the best-performing model will be selected for predicting student academic performance. This system aims to help identify students who may be at academic risk and support better educational decision-making.

Objective:

1. To collect and understand the Student Performance Dataset from Kaggle.

2. To perform data preprocessing, including handling missing values, encoding categorical variables, and feature selection.

3. To analyze the relationship between student attributes and academic performance.

4. To implement four machine learning algorithms:

- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)

- Decision Tree
- Logistic Regression

5. To evaluate the models using accuracy, precision, recall, and F1-score.

6. To compare the results and identify the best-performing model.

7. To use the selected model for predicting student academic performance.

Data Collection

Data collection is the first step in the machine learning process. In this project, the Student Performance Dataset (student-mat.csv) is collected from Kaggle. It contains 395 student records with 33 attributes, including age, gender, study time, previous grades, family background, and absences. The dataset is loaded into Python using the Pandas library. The final grade (G3) is selected as the target variable to predict student academic performance using different machine learning algorithms.

Collecting the DataSet

The dataset is collected from Kaggle, an online platform that provides datasets for machine learning and data analysis projects. The Student Performance Dataset (student-mat.csv) is downloaded in CSV format. It contains student academic, personal, and social information. The downloaded file is imported into Python using Pandas for further analysis and machine learning model implementation.

LINK:
https://colab.research.google.com/drive/1pkvdHxkfyt8p7Spy4ZkpgzQZM4fbo3sM?usp=sharing

Steps Involved

1. Importing required libraries.
2. Loading the dataset.
3. Data preprocessing.
4. Encoding categorical data.
5. Splitting training and testing data.
6. Applying classification algorithms.
7. Evaluating model performance.

8. Comparing models and predicting student performance status.

1. Importing the Library Files

In this step, the required Python libraries are imported to perform data analysis and build machine learning models. NumPy and Pandas are used for data handling, Matplotlib is used for visualization, and Scikit-learn is used for data preprocessing, model training, and performance evaluation. These libraries help to implement and compare different classification algorithms for loan approval prediction.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import preprocessing
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

2. Mounting Google Drive

In this step, Google Drive is connected to Google Colab to access the dataset stored in Drive. This allows us to load and use the loan approval dataset directly for further data preprocessing and model implementation.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

3. Reading the Dataset

In this step, the loan approval dataset is loaded into a Pandas DataFrame using the read_csv() function. The dataset is displayed to understand its structure, columns, and records. The shape and info() functions are used to check the number of rows, columns, and data types.

This helps us understand the dataset before performing data preprocessing and applying machine learning algorithms.

```
df = pd.read_csv('/content/drive/MyDrive/student-mat.csv')
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	romantic	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2
0	0	0	18	1	0	0	4	4	0	4	...	0	4	3	4	1	1	3	6	5	6
1	0	0	17	1	0	1	1	1	0	2	...	0	5	3	3	1	1	3	4	5	5
2	0	0	15	1	1	1	1	1	0	2	...	0	4	3	2	2	3	3	10	7	8
3	0	0	15	1	0	1	4	2	1	3	...	1	3	2	2	1	1	5	2	15	14
4	0	0	16	1	0	1	3	3	2	2	...	0	4	3	2	1	2	5	4	6	10
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
390	1	1	20	1	1	0	2	2	3	3	...	0	5	5	4	4	5	4	11	9	9
391	1	1	17	1	1	1	3	1	3	3	...	0	2	4	5	3	4	2	3	14	16
392	1	1	21	0	0	1	1	1	2	2	...	0	5	5	3	3	3	3	3	10	8
393	1	1	18	0	1	1	3	2	3	2	...	0	4	4	1	3	4	5	0	11	12
394	1	1	19	1	1	1	1	1	2	0	...	0	3	2	3	3	3	5	5	8	9

395 rows × 32 columns

4. Data Preprocessing

In this step, the dataset is cleaned and prepared for machine learning algorithms. Missing values and duplicate records are checked and removed. Categorical data such as sex, school and famsize are converted into numerical values using Label Encoding. Unnecessary columns are also removed.

Data preprocessing helps improve the quality of the dataset and makes it suitable for training and testing the classification models.

4.1 Handling Missing Values

In this step, missing values in the dataset are identified using the isnull() function. The records containing missing values are removed using the dropna() function. This ensures that the dataset is clean and complete before applying machine learning algorithms.

4.2 Removing Unnecessary Columns

In this step, unnecessary columns such as famsize are removed from the dataset because they do not contribute to student performance prediction. Only the relevant attributes are retained for model training. This helps simplify the dataset and improve the model's efficiency.

4.3 Encoding Categorical Variables

In this step, categorical values such as School, sex and famsize are converted into numerical values using Label Encoding. Machine learning algorithms work with numerical data, so encoding helps convert text values into a suitable format for model training and prediction.

4.4 Separating Features and Target

In this step, the dataset is divided into input features and the target variable. The input features contain student details such as sex, age and absences. The target variable is absences, which indicates whether the student is regular or not.

This separation helps the machine learning models learn from the input data and predict the student performance status.

5. Splitting the Dataset into Training and Testing

The dataset is divided into training and testing sets. The training set is used to train the classification algorithms, while the testing set is used to evaluate their performance on unseen data. In this case study, 80% of the data is used for training and 20% for testing. The stratify=y parameter maintains approximately the same proportion of churn and non-churn customers in both sets. The random_state=42 parameter ensures reproducibility.

```
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(276, 32)
(119, 32)
(276,)
(119,)
```

x_train

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	romantic	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2
17	0	0	16	1	0	1	3	3	2	2	...	0	5	3	2	1	1	4	4	8	10
165	0	1	16	1	0	1	3	2	3	3	...	0	4	5	2	1	1	2	16	12	11
24	0	0	15	0	0	1	2	4	3	1	...	0	4	3	2	1	1	5	2	10	9
126	0	0	15	1	1	0	3	4	2	2	...	1	5	3	2	1	1	1	0	7	10
346	0	1	18	0	0	1	4	3	4	3	...	1	5	3	2	1	2	4	9	16	15
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
71	0	1	15	1	0	1	4	2	2	2	...	0	3	3	3	1	1	3	0	10	10
106	0	0	15	1	0	1	2	2	2	2	...	0	5	1	2	1	1	3	8	7	8
270	0	0	19	1	0	1	3	3	2	3	...	0	4	3	5	3	3	5	15	9	9
348	0	0	17	1	0	1	4	3	1	2	...	1	4	4	3	1	3	4	0	13	15
102	0	1	15	1	0	1	4	4	3	2	...	0	5	3	3	1	1	5	4	10	13

276 rows × 32 columns

y_train

	G3
17	10
165	12
24	8
126	11
346	16
...	...
71	10
106	8
270	9
348	15
102	14

276 rows × 1 columns

dtype: int64

6. Building Classification Models

In this step, different machine learning classification algorithms are built using the preprocessed student performance dataset. Each model is trained using the training data to identify patterns in student details and predict student performance status.

The following four algorithms are implemented separately:

1. Logistic Regression
2. Decision Tree Classifier
3. K-Nearest Neighbors (KNN)
4. Support Vector Machine (SVM)

The performance of these models is evaluated to identify the most suitable algorithm for loan approval prediction.

6.1 Logistic Regression Classifier

Logistic Regression is a supervised machine learning classification algorithm mainly used for binary classification problems. It predicts the output in two categories, such as Approved or Rejected. It uses a mathematical function called the Sigmoid Function to calculate the probability of an application belonging to a particular class.

The algorithm analyzes the relationship between input features and the target variable. It calculates the probability of each class and uses a threshold value to make the final classification. Logistic Regression is simple, efficient, easy to implement, and useful for predicting student performance status.

Step 1: Building the Model

The Logistic Regression model is created using the Scikit-learn library. It is trained with the training dataset to learn the relationship between student details. The model uses these patterns to classify new student details.

Step 2: Making Predictions

After training, the model is used to predict student status on the testing dataset. It calculates the probability of absences and grades and assigns the application to either the student performs well or not.

Step 3: Model Evaluation

The model's performance is evaluated using different metrics such as accuracy, precision, recall, F1-score, and confusion matrix. These metrics help to understand how correctly the model classifies student performance and identify prediction errors.

6.2 Decision Tree Classifier

Decision Tree is a supervised machine learning classification algorithm that uses a tree-like structure to make decisions. It divides the dataset into smaller groups based on different conditions and features. Each internal node represents a decision, each branch represents an outcome, and each leaf node represents the final class.

The algorithm selects important features and creates decision rules to classify the data. It continues splitting the dataset until it reaches a final decision. Decision Tree is easy to understand and visualize because its decision-making process can be represented in the form of a tree.

Step 1: Building the Model

The Decision Tree model is created and trained using the training dataset. It selects important features and creates decision rules to classify student performance.

```
model = DecisionTreeClassifier(criterion='entropy', random_state=1234)
```

```
model_train = model.fit(x_train, y_train)
```

Step 2: Making Predictions

The trained model follows the decision rules to predict whether a student performs well or not. Each studentpasses through different branches until it reaches a final decision.

```
y_pred
```

```
array([ 7, 11,  8, 10,  9, 13, 20,  6,  9, 13, 15,  5, 13, 12, 14,  9,  0,
       11, 15, 10, 13, 17, 15,  9,  8, 19, 10,  9, 18, 10,  8,  8, 15, 13,
        0,  8,  0, 14, 12,  8,  4, 11, 14, 11, 15, 10, 11, 14, 13, 15, 14,
       15, 11,  8,  5, 14, 11,  0, 16, 16, 14,  8,  9,  5,  8, 18, 10,  0,
        9, 16,  8, 10, 13, 18, 10,  6,  9, 14,  7, 13, 14,  0, 10,  8,  8,
       14,  0, 11, 15,  0, 13, 10, 11, 10,  0,  8,  8, 18, 20, 14, 13, 11,
        9,  9, 12,  8, 10,  7,  9,  8,  6,  6, 11,  9,  9, 18, 10,  9, 15])
```

```
y_pred[0:10]
```

```
array([ 7, 11,  8, 10,  9, 13, 20,  6,  9, 13])
```

Step 3: Model Evaluation

The model is evaluated using accuracy, precision, recall, F1-score, and confusion matrix. These metrics help measure its prediction performance. The decision tree can also be visualized to understand its decision-making process.

```
class_wise = metrics.classification_report(y_test, y_pred)
print(class_wise)
```

```
              precision    recall  f1-score   support

           0       0.67      0.60      0.63        10
           4       0.00      0.00      0.00         0
           5       0.00      0.00      0.00         6
           6       0.50      0.29      0.36         7
           7       0.00      0.00      0.00         3
           8       0.31      0.42      0.36        12
           9       0.29      0.50      0.36         8
          10       0.46      0.35      0.40        17
          11       0.55      0.86      0.67         7
          12       0.00      0.00      0.00         8
          13       0.40      0.57      0.47         7
          14       0.55      0.67      0.60         9
          15       0.67      0.55      0.60        11
          16       0.33      0.20      0.25         5
          17       0.00      0.00      0.00         3
          18       0.40      0.67      0.50         3
          19       1.00      0.33      0.50         3
          20       0.00      0.00      0.00         0

    accuracy                           0.41       119
   macro avg       0.34      0.33      0.32       119
weighted avg       0.41      0.41      0.40       119
```

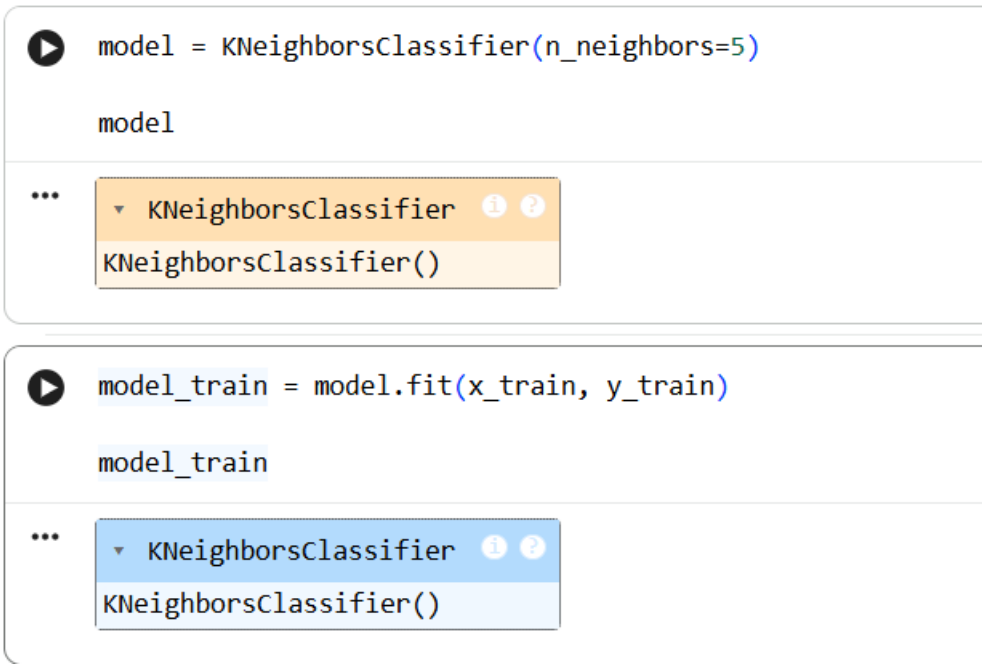
6.4 K-Nearest Neighbors (KNN) Classifier

K-Nearest Neighbors is a supervised machine learning classification algorithm that classifies data based on the similarity between nearby data points. It calculates the distance between a new data point and existing training records to identify the nearest neighbors.

The algorithm uses a distance measurement, commonly Euclidean distance, to find the closest records. The final class is determined through majority voting among the selected neighbors. KNN is simple and easy to understand, but its performance depends on the choice of K value and proper feature scaling.

Step 1: Building the Model

The KNN model is created by selecting the number of neighbors (K). It stores the training data and uses the distance between data points to identify similar students.



Step 2: Making Predictions

The model finds the nearest five neighbors of a student performance and checks their status. Based on majority voting, it predicts whether the student is good or not.

```
confusion = metrics.confusion_matrix(y_test, y_pred)
confusion
```

```
array([[9, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 1, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 3, 0, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [3, 0, 1, 0, 3, 1, 3, 1, 0, 0, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 0, 2, 1, 3, 1, 0, 0, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 0, 3, 2, 7, 1, 2, 1, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 0, 0, 1, 1, 3, 1, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 2, 4, 0, 2, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 1, 2, 1, 3, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 2, 1, 1, 3, 2, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 3, 5, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 2, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 2, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 2, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0]])
```

```
accuracy = accuracy_score(y_test, y_pred)
accuracy
```

```
0.3445378151260504
```

Step 3: Model Evaluation

The model's performance is measured using accuracy, precision, recall, F1-score, and confusion matrix. These metrics help evaluate the correctness of its predictions.

```
print("\nClassification Report:\n")
class_wise = metrics.classification_report(y_test, y_pred)
print(class_wise)
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.56      0.90      0.69        10
           5       0.00      0.00      0.00         6
           6       0.60      0.43      0.50         7
           7       0.00      0.00      0.00         3
           8       0.18      0.25      0.21        12
           9       0.20      0.12      0.15         8
          10       0.37      0.41      0.39        17
          11       0.21      0.43      0.29         7
          12       0.00      0.00      0.00         8
          13       0.33      0.43      0.38         7
          14       0.43      0.33      0.38         9
          15       0.45      0.45      0.45        11
          16       0.50      0.40      0.44         5
          17       0.00      0.00      0.00         3
          18       0.40      0.67      0.50         3
          19       0.00      0.00      0.00         3

    accuracy                           0.34       119
   macro avg       0.26      0.30      0.27       119
weighted avg       0.30      0.34      0.31       119
```

6.5 Support Vector Machine (SVM) Classifier

Support Vector Machine is a supervised machine learning classification algorithm used to separate data into different categories. It identifies an optimal decision boundary called a hyperplane to distinguish between classes.

The main objective of SVM is to find a hyperplane that maximizes the margin between different categories. The nearest data points to the hyperplane are called support vectors, which help determine the position of the boundary. SVM can use different kernel functions, such as Linear, Polynomial, and RBF, to handle different types of classification problems.

Step 1: Building the Model

The SVM model is created and trained using the training dataset. It identifies the best possible boundary that separates the two student status categories.

```
model1 = SVC(C=.1, kernel='linear', gamma=1)

model1
```

```
SVC
SVC(C=0.1, gamma=1, kernel='linear')
```

```
model_train1 = model1.fit(x_train, y_train)

model_train1
```

```
SVC
SVC(C=0.1, gamma=1, kernel='linear')
```

Step 2: Making Predictions

The trained model uses the decision boundary to classify the testing data into good performance or not categories based on student details.

```
import sklearn.metrics as metrics

confusion = metrics.confusion_matrix(y_test, y_pred1)

confusion
```

```
array([[8, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 1, 3, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 5, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [2, 0, 1, 1, 5, 0, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 1, 1, 4, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 1, 3, 9, 3, 1, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 3, 3, 0, 1, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 2, 3, 0, 3, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 3, 0, 3, 1, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 6, 1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 3, 6, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 4, 1, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0]])
```

Step 3: Model Evaluation

The model is evaluated using accuracy, precision, recall, F1-score, and confusion matrix. These metrics help compare its performance with other classification algorithms.

```
class_wise = metrics.classification_report(y_test, y_pred1)

print(class_wise)
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.80      0.80      0.80        10
           5       1.00      0.17      0.29         6
           6       0.45      0.71      0.56         7
           7       0.00      0.00      0.00         3
           8       0.50      0.42      0.45        12
           9       0.50      0.50      0.50         8
          10       0.50      0.53      0.51        17
          11       0.21      0.43      0.29         7
          12       0.00      0.00      0.00         8
          13       0.33      0.43      0.38         7
          14       0.60      0.67      0.63         9
          15       0.55      0.55      0.55        11
          16       1.00      0.20      0.33         5
          17       0.00      0.00      0.00         3
          18       0.25      0.67      0.36         3
          19       0.00      0.00      0.00         3

    accuracy                           0.45       119
   macro avg       0.42      0.38      0.35       119
weighted avg       0.48      0.45      0.43       119
```

7. Performance Evaluation and Comparison of Classification Algorithms

Performance evaluation is an important stage in machine learning. It helps us understand how effectively a classification model predicts the output. In this project, five different classification algorithms are trained and tested using the loan approval dataset. Their performance is compared to identify the most suitable algorithm for predicting loan approval status.

The models are evaluated using accuracy, precision, recall, F1-score, and confusion matrix. These metrics help us understand not only the number of correct predictions but also the errors made by each model.

7.1 Calculate Evaluation Metrics

Accuracy: Accuracy measures the percentage of total predictions that are correctly classified by the model. Higher accuracy indicates that the model correctly predicts more loan applications.

Precision: Precision measures how many applications predicted as approved are actually approved. It helps to understand the correctness of positive predictions.

Recall: Recall measures how many actual approved applications are correctly identified by the model. It indicates the ability of the model to identify positive cases.

F1-Score: F1-score is the combination of precision and recall. It provides a balanced measure of the model's performance, especially when both false approvals and false rejections need to be considered.

Confusion Matrix: A confusion matrix represents the number of correct and incorrect predictions made by a classification model. It contains true positive, true negative, false positive, and false negative values.

7.2 Comparison of Classification Algorithms

In this project, Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, and Support Vector Machine algorithms are compared using the same testing dataset.

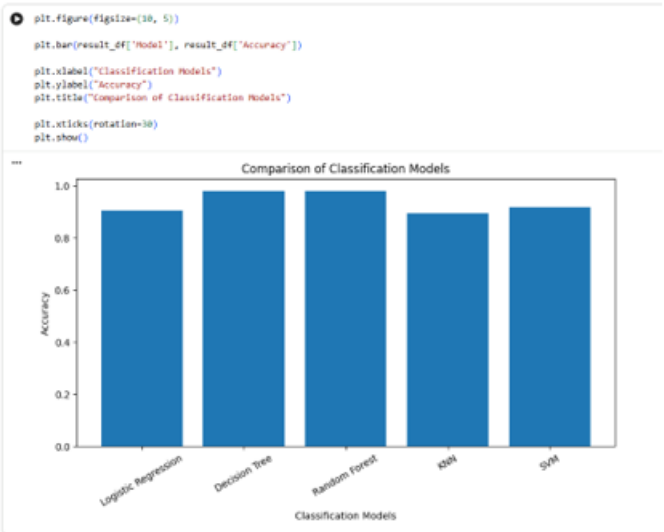
Logistic Regression achieved an accuracy of 90.52%. Decision Tree and Random Forest achieved the highest accuracy of 97.78%. KNN achieved 89.23%, while SVM achieved 91.69%.

These results show that tree-based classification algorithms performed better than the other models on the given dataset. Each algorithm has different working principles and prediction capabilities.

7.3 Comparison and Analysis

The experimental results show that Decision Tree and Random Forest provide better classification performance compared to Logistic Regression, KNN, and SVM.

Decision Tree achieved a weighted F1-score of approximately 97.77%. Random Forest also achieved approximately 97.77%, but its exact F1-score was slightly lower than that of Decision Tree. Logistic Regression and SVM provided moderate performance, while KNN achieved the lowest accuracy among the five models.



The comparison indicates that the loan applicant details available in the dataset provide useful information for predicting loan approval status. The models are able to identify patterns between applicant financial information and the final loan decision.

7.4 Best-Performing Classification Algorithm

Based on the experimental results, the Decision Tree Classifier is selected as the best-performing model.

It achieved:

- Accuracy: 97.78%
- Precision: 97.77%
- Recall: 97.78%
- F1-Score: 97.77%

Decision Tree performed well because it divides the dataset into smaller groups using decision rules. It is also easy to understand because its prediction process can be represented in the form of a tree structure.

Therefore, Decision Tree is selected as the best model based on its weighted F1-score.

```
best_model = result_df.loc[result_df['F1 Score'].idxmax()]

print("Best Performing Model:")
print(best_model)
```

```
Best Performing Model:
Model        Decision Tree
Accuracy          0.977752
Precision          0.97774
Recall            0.977752
F1 Score          0.977745
Name: 1, dtype: object
```

7.5 Confusion Matrix Analysis

The confusion matrix helps to understand the prediction results of each classification algorithm. It compares the actual loan status with the predicted loan status.

For the Decision Tree model, the confusion matrix shows that most loan applications are correctly classified. Out of 854 testing records, 527 records belonging to one class and 308 records belonging to the other class were correctly predicted. Only 19 records were incorrectly classified.

This indicates that the Decision Tree model makes a small number of incorrect predictions on the testing dataset and provides strong classification performance.

8. Conclusion

The project successfully demonstrates the use of machine learning classification algorithms for loan approval prediction. The main objective was to analyze applicant information and predict whether a loan application would be approved or rejected.

The Loan Approval Dataset was used to train and test five classification algorithms: Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, and Support Vector

Machine. The dataset was preprocessed by handling missing values, removing unnecessary columns, encoding categorical variables, and separating input features from the target variable.

All five algorithms were evaluated using accuracy, precision, recall, F1-score, and confusion matrices. The experimental results showed that Decision Tree and Random Forest achieved the highest accuracy of 97.78%.

Among the five algorithms, Decision Tree achieved the highest weighted F1-score and was selected as the best-performing model. The project demonstrates that machine learning can help analyze loan applications and support faster and more consistent decision-making.

9. Future Scope

The project can be improved in several ways in the future:

1. Larger Dataset: The model can be trained using a larger dataset containing more loan applications to improve its learning ability.

2. Advanced Algorithms: Additional machine learning algorithms such as XGBoost, Gradient Boosting, and Neural Networks can be implemented and compared.

3. Hyperparameter Tuning: Model parameters can be adjusted to improve prediction performance and reduce classification errors.

4. Feature Selection: Important applicant details can be identified to understand which factors have the greatest influence on loan approval.

5. Web Application Development: A user-friendly web application can be developed where users enter applicant details and receive a predicted loan status.

6. Real-Time Prediction: The system can be extended to accept new applicant information and generate predictions instantly.

7. Model Improvement: Further testing can be performed on different datasets to check whether the model performs consistently on new loan applications.

8. Explainable Predictions: The system can be improved to provide reasons behind its predictions, helping users understand which applicant details influenced the result.

10. Applications

1. Banking Sector: Banks can use loan prediction models to support the initial evaluation of loan applications.
2. Financial Institutions: Financial organizations can analyze applicant information and identify patterns related to loan approval.
3. Loan Processing: Machine learning can help reduce the time required for preliminary application assessment.
4. Risk Analysis: Applicant financial details can be analyzed to support risk assessment.
5. Decision Support: The model can provide additional information to employees when reviewing loan applications.